New Features in C# 6

http://blogs.msdn.com/b/csharpfaq/archive/2014/11/20/new-features-in-c-6.aspx

C# 6 is the version of C# that ships with VS 2015 Preview. The philosophy behind this version is straightforward: improve simple everyday coding scenarios, without adding much conceptual baggage. The features should make code lighter without making the language heavier.

Some features are mostly syntactic sugar (*expression-bodied members*, *string interpolation*, *nameof*), some complete what felt a bit like unfinished work (*getter-only auto-properties*, *await in catch and finally*) or expand existing features in a natural way (*using static*, *index initializers*). Some expose expressiveness from IL that C# didn't previously surface (*exception filters*, *parameterless constructors in structs*). Some are almost just "bug fixes" to the design, and you'll hardly know that you're making use of them (*improved overload resolution*, *extension Add methods in collection initializers*).

In the following, I'll take you through each of these language features. We'll start at the expression level, and work our way "outwards". We're quite confident that this is the set that will end up in the shipping version, but we already know that some of them will change a little. I'll let you know when we get there.

Expression-level features

nameof expressions are a new form of string literal that require *more* syntax and are *more* restricted than the existing kinds. What's not to like?

Actually there's a lot to like. Oftentimes you need to provide a string that names some program element: when throwing an `ArgumentNullException` you want to name the guilty argument; when raising a `PropertyChanged` event you want to name the property that changed, etc.

Using ordinary string literals for this purpose is simple, but error prone. You may spell it wrong, or a refactoring may leave it stale. `nameof` expressions are essentially a fancy kind of string literal where the compiler checks that you have something of the given name, and Visual Studio knows what it refers to, so navigation and refactoring will work:

```
(if x == null) throw new ArgumentNullException(nameof(x));
```

You can put more elaborate dotted names in a `nameof` expression, but that's just to tell the compiler where to look: only the final identifier will be used:

```
WriteLine(nameof(person.Address.ZipCode)); // prints "ZipCode"
```

Note: there are small design changes to nameof since the Preview was built. In the preview, dotted expressions like in the last example, where person is a variable in scope, are not allowed. Instead you have to dot in through the type.

String interpolation lets you more easily format strings. `String.Format` and its cousins are very versatile, but their use is somewhat clunky and error prone. Particularly unfortunate is the use of numbered placeholders like `{0}` in the format string, which must line up with separately supplied arguments:

```
var s = String.Format("{0} is {1} year{{s}} old", p.Name, p.Age);
```

String interpolation lets you put the expressions right in their place, by having "holes" directly in the string literal:

```
var s = "\{p.Name} is \{p.Age} year{s} old";
```

Just as with `String.Format`, optional alignment and format specifiers can be given:

```
var s = "\{p.Name,20} is \{p.Age:D3} year{s} old";
```

The contents of the holes can be pretty much any expression, including even other strings:

```
var s = "\{p.Name} is \{p.Age} year\{(p.Age == 1 ? "" : "s")} old";
```

Notice that the conditional expression is parenthesized, so that the '`: "s"`' doesn't get confused with a format specifier.

Note: This describes the syntax that works in the Preview. However, we've decided to change the syntax, to even better match that of format strings. In a later release you'll see interpolated strings written like this:

```
var s = $"{p.Name,20} is {p.Age:D3} year{{s}} old";
```

Also we are adding a scheme so that you can influence the formatting behavior; e.g. to use Invariant Culture. More on this in a later post.

Null-conditional operators address many of the situations where code tends to drown in null-checking. They let you access members and elements only when the receiver is not-null, providing a null result otherwise:

```
int? length = customers?.Length; // null if customers is null
Customer first = customers?[0];  // null if customers is null
```

The null-conditional operator is conveniently used together with the null-coalescing operator `` `??` `` to provide a default value for when the outcome is null:

```
int length = customers?.Length ?? 0; // 0 if customers is null
```

The null-conditional operator exhibits short-circuiting behavior, where an immediately following chain of member accesses, element accesses and invocations will only be executed if the original receiver was not null:

```
int? first = customers?[0].Orders.Count(); // if customers is null, the result is null
```

This example is essentially equivalent to:

```
int? first = (customers != null) ? customers[0].Orders.Count() : null;
```

Except that `customers` is only evaluated once. None of the member accesses, element accesses and invocations immediately following the `?` are executed unless `customers` has a non-null value.

Of course null-conditional operators can themselves be chained, in case there is a need to check for null more than once in a chain:

```
int? first = customers?[0].Orders?.Count();
```

Note that an invocation (a parenthesized argument list) cannot immediately follow the `?` operator – that would lead to too many syntactic ambiguities. Thus, the straightforward way of calling a delegate *only* if it's there does not work:

```
if (predicate?(e) ?? false) { … } // Error!!
```

However, you can do it via the `Invoke` method on the delegate:

```
if (predicate?.Invoke(e) ?? false) { … }
```

We expect that a very common use of this pattern will be for triggering of events:

```
PropertyChanged?.Invoke(this, args);
```

This is an easy and thread-safe way to check for null before you trigger an event. The reason it's thread-safe is that the feature evaluates the left-hand side only once, and keeps it in a temporary variable.

Index initializers extend object initializers so that you can now initialize not only members but also indices of the newly created object:

```
var numbers = new Dictionary<int, string> {
    [7] = "seven",
    [9] = "nine",
    [13] = "thirteen"
};
```

A common use for this would be for creating Json objects declaratively in one expression, rather than having to first create the object and then assign into it in separate statements.

Extension Add methods are now observed in collection initializers; we used to only recognize *instance* methods called `Add`. Just a design fix; VB already does this.

Overload resolution has been improved in a few ways, which will likely result in more things just working the way you'd expect them to. The improvements all relate to "betterness" – the way the compiler decides which of two overloads is better for a given argument.

One place where you might notice this (or rather stop noticing a problem!) is when choosing between overloads taking nullable value types. Another is when passing method groups (as opposed to lambdas) to overloads expecting delegates. The details aren't worth expanding on here – just wanted to let you know!

Statement-level features

Exception filters are a CLR capability that is exposed in Visual Basic and F#, but hasn't been in C# – until now. This is what they look like:

```
try { … }
catch (MyException e) if (myfilter(e))
{
    …
}
```

If the parenthesized expression after '`if`' evaluates to true, the catch block is run, otherwise the exception keeps going.

Exception filters are preferable to catching and rethrowing because they leave the stack unharmed. If the exception later causes the stack to be dumped, you can see where it originally came from, rather than just the last place it was rethrown.

It is also a common and accepted form of "abuse" to use exception filters for side effects; e.g. logging. They can inspect an exception "flying by" without intercepting its course. In those cases, the filter will often be a call to a false-returning helper function which executes the side effects:

```
private static bool Log(Exception e) { /* log it */ ; return false; }
…
try { … }
catch (Exception e) if (Log(e)) {}
```

Await in catch and finally blocks has been disallowed until now. We'd somehow convinced ourselves that it wasn't possible to implement, but now we've figured it out, so apparently it wasn't impossible after all.

This has actually been a significant limitation, and people have had to employ unsightly workarounds to compensate. That is no longer necessary:

```
Resource res = null;
try
{
    res = await Resource.OpenAsync(…); // You could always do this.
    …
}
catch(ResourceException e)
{
    await Resource.LogAsync(res, e); // Now you can do this …
}
finally
{
    if (res != null) await res.CloseAsync(); // … and this.
}
```

The implementation in the compiler *is* quite complicated, but you don't have to worry about that. That's the whole point of having async in the language: so that most of the complicated stuff is left to the compiler.

Member declaration features

Auto-property initializers are now enabled, and are similar to initializers on fields:

```
public class Customer
{
    public string First { get; set; } = "Jane";
    public string Last { get; set; } = "Doe";
}
```

The initializer directly initializes the backing field; it doesn't work through the setter of the auto-property. The initializers are executed in order as written, just as – and along with – field initializers.

Just like field initializers, auto-property initializers cannot reference '`this`' – after all they are executed before the object is properly initialized.

Getter-only auto-properties allow you to omit a setter on an auto-property:

```
public class Customer
{
public string First { get; } = "Jane";
public string Last { get; } = "Doe";
}
```

The backing field of a getter-only auto-property is implicitly declared as `readonly` (though this matters only for reflection purposes). It can be initialized through an initializer on the property as in the example above, or it can be assigned in the declaring type's constructor body, which causes the value to be assigned directly to the underlying field:

```
public class Customer
{
    public string Name { get; };
   public Customer(string first, string last)
    {
        Name = first + " " + last;
    }
}
```

This is about expressing types more concisely, but note that it also removes an important difference in the language between mutable and immutable types: auto-properties were a shorthand available only if you were willing to make your class mutable, and so the temptation to default to that was great. Now, with getter-only auto-properties, the playing field has been leveled between mutable and immutable.

Expression-bodied function members allow methods, properties and other kinds of function members to have bodies that are expressions instead of statement blocks, just like with lambda expressions.

Methods as well as user-defined operators and conversions can be given an expression body by use of the "lambda arrow":

```
public Point Move(int dx, int dy) => new Point(x + dx, y + dy);
public static Complex operator +(Complex a, Complex b) => a.Add(b);
public static implicit operator string(Person p) => "\{p.First} \{p.Last}";
```

The effect is exactly the same as if the methods had had a block body with a single return statement.

For void returning methods – and `Task` returning async methods – the arrow syntax still applies, but the expression following the arrow must be a statement expression (just as is the rule for lambdas):

```
public void Print() => Console.WriteLine(First + " " + Last);
```

Properties and indexers can have getters and settersgetter-only properties and indexers can have an expression body:

```
public string Name => First + " " + Last;
public Customer this[long id] => store.LookupCustomer(id);
```

Note that there is no `get` keyword: it is implied by the use of the expression body syntax.

Parameterless constructors in structs are now allowed:

```
struct Person
{
    public string Name { get; }
    public int Age { get; }
    public Person(string name, int age) { Name = name; Age = age; }
    public Person() : this("Jane Doe", 37) { }
}
```

The expression '`new Person()`' will execute the declared constructor instead of the standard behavior of providing the default value. Note, however, that '`default(Person)`' will still produce the default value, as will '`new Person[…]`' for each of the array elements. In those cases the constructor is *not* executed: It's only when you explicitly use '`new`' on the struct type.

Import features

using static is a new kind of using clause that lets you import static members of types directly into scope.

In the Preview it looks like this:

```
using System.Console;
using System.Math;

class Program
{
    static void Main()
    {
        WriteLine(Sqrt(3*3 + 4*4));
    }
}
```

This is great for when you have a set of functions related to a certain domain that you use all the time. `System.Math` would be a common example of that.

Note: The Preview will only import members of static classes. We have since changed the design in several ways:

1. *The syntax will be more different from current using clauses – it will have the keywords "*`using static`*".*
2. *Members of non-static types can be imported, including structs and en ums*
3. *Members of VB modules and F# top level functions can be imported*
4. *Extension methods will not be imported into top-level scope; instead they will show up as extension methods. This gives a way to import extension methods from just a single class, something which wasn't possible before.*